State 方面，我们采用不同于其他论文的 augmented state $s_t = (x_t, \xi_t, \tau_t)$，包括$x_t$：当前的带噪声图/带噪潜变量。$\xi_t$：历史累计状态(integral state)，吸收了过去所有步中缝合失败的误差，由 reward function 提供。τ_t：趋势状态(derivative state)，反映误差的变化程度。其中$\xi_t = \sum_{k=T}^{t} \nabla\mathcal{R}_k(x_k)$，$\tau_t = \nabla\mathcal{R}_t(x_t) - \nabla\mathcal{R}_{t+1}(x_{t+1})$，R 是可以计算的 reward function（待确定）。

Transition Probability 方面，我们采用 DDIM 的框架（代码实现层面），因为它的确定性采样轨迹相比于在每一步注入随机噪声的 DDPM 而言更加平滑，能让我们的 PID 控制器在更稳定的流形上施加牵引力。而在理论部分，我们采用 song yang 的受控 SDE 视角框架来进行证明。此外，我的 transition probability 完全由预训练模型 standard stable diffusion v1.5 的权重θ决定，而且这个权重完全冻结，不能改变。我们用的 stable diffusion v1.5 只能生成整张随机的图，完全不知道 mask 是什么。而我们只需要定义 MDP 和外挂一个 PID policy 就可以把它驯化成一个强大 inpainting 模型。如果我们找一个已经可以做 inpainting 的模型，那么会降低我们的 contribution。我们其实并没有训练模型去 inpainting 的能力，而是修改它的行进路线（改变潜变量的生成轨迹）。

$$\boldsymbol{x}_{t-1} = \sqrt{\alpha_{t-1}} \underbrace{\left(\frac{\boldsymbol{x}_t - \sqrt{1-\alpha_t}\epsilon_\theta^{(t)}(\boldsymbol{x}_t)}{\sqrt{\alpha_t}} \right)}_{\text{“ predicted } \boldsymbol{x}_0\text{”}} + \underbrace{\sqrt{1-\alpha_{t-1}-\sigma_t^2} \cdot \epsilon_\theta^{(t)}(\boldsymbol{x}_t)}_{\text{“direction pointing to } \boldsymbol{x}_t\text{”}} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}} \tag{12}$$

Action 方面，我们与传统的论文去把下一幅噪声图作为 action 不同，我们的 action 不是一张图片，也不是一组神经网络权重。**它是一个高维的连续梯度张量（Gradient Tensor / Guidance Vector）。**它的维度必须和潜变量x_t的维度相同，并且处在同一个空间。它的本质是一个方向向量，每一个数值代表了对应隐变量通道上的一个拉力或位移指令

- **物理方程中的位置：** 正如我们在前面的受控 SDE 中讨论的，这个动作被直接**相加**到了 U-Net 的单步去噪结果上：

$$x_{t-1} = \underbrace{\mathrm{UNet}(x_t, t)}_{\text{环境自身的盲目演化}} + \underbrace{a_t}_{\text{你施加的上帝之手}}$$

在上面的图中，U-Net(x_t, t)这一项采用 DDIM 的降噪规则去演化出初步的$\hat{x}_{t-1}$，然后这一项加上a_t之后才是我们最后更新的新噪声图x_t。即：$x_{t-1} = \hat{x}_{t-1} + a_t$，$\hat{x}_{t-1}$由 DDIM 的降噪规则获得。

$$\boldsymbol{x}_{t-1} = \sqrt{\alpha_{t-1}} \underbrace{\left(\frac{\boldsymbol{x}_t - \sqrt{1-\alpha_t}\epsilon_\theta^{(t)}(\boldsymbol{x}_t)}{\sqrt{\alpha_t}} \right)}_{\text{“ predicted } \boldsymbol{x}_0\text{”}} + \underbrace{\sqrt{1-\alpha_{t-1}-\sigma_t^2} \cdot \epsilon_\theta^{(t)}(\boldsymbol{x}_t)}_{\text{“direction pointing to } \boldsymbol{x}_t\text{”}} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}} \tag{12}$$

U-Net 代表了自然的引力（让图像看起来真实），而a_t是一个精确计算出的人造牵引力（让图像符合边界约束）。概括来说，Action 是一个由 PID 控制器实时计算出的、与潜空间同维度的引导梯度场（Guidance Vector Field）。它像隐形的模具一样，每一步都在微调生成图像的形状，逼迫其长成完美吻合边界的样子。

Reward 方面，我们考虑了很多种衡量指标，包括 L2 距离（将用在 2D 的 toy example 上）、LPIPS 特征距离、边界像素的梯度差异（total variation）、DINO 语义特征差异、U-Net 内部特征图的空间平滑度。具体采用哪个指标，我们后续再选择。目前在 2D 的 toy example 上，我们先采用 L2 距离；在高维图像上，我们先采用 LPIPS 特征距离+边界像素的梯度差异（total variation）的组合，即

$$\mathcal{R}_t^{Total}(s_t) = \mathcal{R}_t^{LPIPS}(\hat{x}_0, y) + \mathcal{R}_t^{TV}(\hat{x}_0)$$

用来保证局部边界一致性和全体图像的合理协调性。对这个$\mathcal{R}_t^{Total}$对当前潜变量x_t求导，得到我们的奖励函数的梯度，即$g_t = \nabla_{x_t}\mathcal{R}_t^{Total}$。$g_t$将用于后续更新$\xi_t$和$\tau_t$。而所有 reward 的计算，都依赖于我们先根据当前的噪声图，通过 Tweedie's formula 来先行预测一下预期的干净图$\widehat{x_0(x_t)}$，然后拿这幅图$\widehat{x_0(x_t)}$，与当前的边界 y（即 condition）输入一个 reward function，得到我们想要的$\mathcal{R}_t^{Total}$。

$$\hat{\boldsymbol{x}}_0 := \mathbb{E}[\boldsymbol{x}_0|\boldsymbol{x}_t] = \frac{1}{\sqrt{\bar{\alpha}(t)}}(\boldsymbol{x}_t + (1-\bar{\alpha}(t))\nabla_{\boldsymbol{x}_t}\log p_t(\boldsymbol{x}_t))$$

Policy 方面，我们采用一种 Non-parametric Adaptive Policy 的方法，包含 PID 自适应调节和 SNR 安全锁。在传统的深度强化学习（如 PPO、DQN）中：接收 Reward→算出全局分数→更新 Policy→利用反向传播，修改神经网络参数θ→θ' →改变 Action：下次遇到同样的状态，因为大脑θ变了，所以做出的动作变了。

而我们的方法：将传统 RL 中参数空间的微调转化为了状态空间的吸收。

1. 接收 Reward：每一步计算边界误差，得到瞬时梯度 $g_t = \nabla_{x_t}\mathcal{R}_t(x_t)$
2. 更新 Policy：我们的 Policy 公式$\pi(s_t)$永远不变（K_p, K_i, K_d写死）。但是，Policy 接收的输入状态s_t会吸收 Reward。具体来说，历史积分状态ξ_t把刚才的误差g_t吞了进去，膨胀了。
3. 改变 Action：当进行到下一步时，虽然x_t的局部图像看起来没怎么变，但因为 Policy 的"记忆区"ξ_t变大了，所以带入 PID 公式后，计算出的动作力a_t发生了剧烈的增强。

总结来说，Policy 不是通过改变自身的代码（参数）来学习，而是通过维持一个内部的记忆账本（积分状态ξ_t）来在线自适应环境的惩罚。具体的 policy 公式如下：

$$a_t = \pi_t(s_t) = f_{SNR}(t) \bullet [K_p\nabla_{x_t}R_t(x_t) + K_i\xi_t + K_d\tau_t]$$

这里面K_p，K_i，K_d都是超参数，我们将找到最佳的超参数值然后写死。而 SNR 锁的具体形式我们将会在之后给出并详细解释。

具体来说，我们的一步更新流程如下：

Step1：在当前时间 t 步，state 是$s_t = (x_t, \xi_{t+1}, \tau_{t+1})$，根据当前 state，我们用预训练

好的 U-Net 根据x_t输出预测的噪声，然后利用 Tweedie's formula 算出预测的干净图的期望$\widehat{x_0(x_t)}$。然后利用$\widehat{x_0(x_t)}$与当前边界 y（即 condition 的条件）来计算当前的 reward（即求我们的 loss function），算出 reward 之后求梯度便可以得到当前的瞬间梯度$g_t = \nabla_{x_t} R_t(x_t)$。

Step 2：现在我们有了g_t之后就可以更新我们 state 中的 integral state 和 derivative state 了，分别更新为$\xi_t = \xi_{t+1} + g_t$和$\tau_t = g_t - g_{t+1}$。

Step 3：然后由我们的 policy 输出 action：$a_t = \pi_t(s_t) = f_{SNR}(t) \bullet [K_p \nabla_{x_t} R_t(x_t) + K_i \xi_t + K_d \tau_t]$。

Step 4：得到 action a_t后，先由预训练模型将x_t进行一步标准的 DDIM 降噪得到$\hat{x}_{t-1}$，然后加上我们施加的 action 作为修正，得到$x_{t-1} = \hat{x}_{t-1} + a_t$，从而获得下一步的噪声图$x_{t-1}$并实现状态的更新，新的 state 为$s_{t-1} = (x_{t-1}, \xi_t, \tau_t)$，然后进行下一个循环。